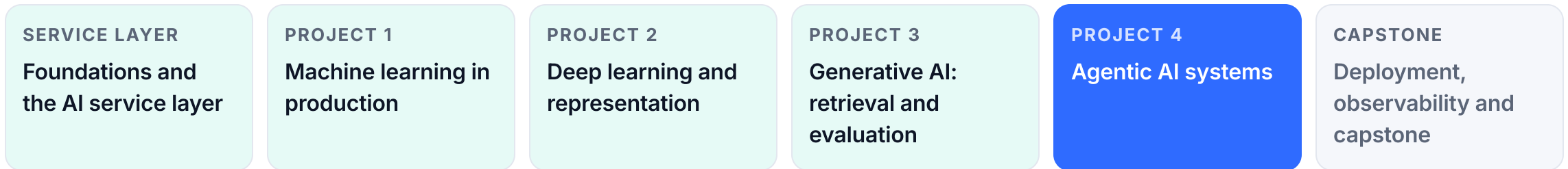


Agentic AI systems

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/agent`, Project 4: an agent that uses the three projects as tools, with a hard limit on what it may do alone. Before the code, a use case in the catalog; after it, a card in the registry, an evaluation by tier, a runtime in your tenant's namespace, a guardrail in front and a gateway in front of its tools.

Three services can score, sort and answer; nobody decides



A priority scorer, a semantic engine and a policy service that answers with citations. Deciding what to do with a ticket is the first job that needs an agent: the path depends on what each tool returns. The capstone promotes it.

Today you build a resolver agent with tools, approvals and a tiered gate

- 1 Register the use case first
- 2 Read the loop before you build it
- 3 The registry validates; a boolean gets a reason
- 4 The loop
- 5 Run it for real, read the trace
- 6 Break it, measure it by tier
- 7 Approve, and the action runs once
- 8 The tiered release gate
- 9 Orchestrator and specialists over HTTP
- 10 Agent cards and the registry
- 11 Deploy to your tenant's runtime
- 12 The guardrail and the tool gateway
- 13 The same evaluation through a framework (AWS, Google Cloud)
- 14 Sample traces for review
- 15 The part's check and commit
- 16 The registry as an MCP server

An agent chooses its next step; a workflow follows the one you wrote

Workflow

LLMs and tools follow predefined code paths.
Project 3's `/ask` is one: retrieve, filter, rerank, answer, validate, in that order, every time.

Agent

LLMs "dynamically direct their own processes and tool usage".
Project 4's resolver is one: the model picks the tool, reads the result, and picks again until it answers or a cap stops it.

NIST's framing for policy: agent systems are "capable of planning and taking autonomous actions that impact real-world systems or environments". The last three words are why this part has an approval gate.

Two open protocols now wire agents to tools and to each other

	MCP, Model Context Protocol	A2A, Agent2Agent protocol
Connects	a host to servers offering tools, resources and prompts	one agent to another agent, each described by an agent card
Current version	spec 2026-07-28, JSON-RPC 2.0 messages	v1.0 on 2026-03-12, v1.0.1 on 2026-05-28
Governed by	Agentic AI Foundation, Linux Foundation, since 2025-12-09	Linux Foundation, since 2025-06-23
In this course	the tool registry as an MCP server behind the tool gateway	the resolver's card, an A2A agent card in AWS Agent Registry

MCP's specification requires explicit user consent before a tool is invoked. Your approval gate is that consent, written in code.

An agent without a written purpose cannot be evaluated, only admired

Use case	State
<code>uc-triage</code> , <code>uc-policy</code> , <code>uc-resolution</code>	approved by support-operations
<code>uc-resolver</code> , <code>uc-orchestrator</code> , <code>uc-router</code>	approved by support-operations
<code>uc-refund</code> , high risk	draft, no agent in code

Each entry also carries its EU AI Act class, the DPIA section and the privacy approver. `make governance-check` beside `make catalog-check`.

Two rules the check reads from code

Every agent in code has an approved entry, or the check fails.

An agent's tools are a subset of its entry's `allowed_tools`, and its reachable data classes are derived from those tools.

An agent is a loop; everything else is guard rails

```
for index in range(max_steps):
    if spent >= budget_usd:          -> BUDGET
    reply = client.complete(messages, tools=specs)
    if not reply.tool_calls:         -> ANSWER
    for call in reply.tool_calls:
        approved = _decide(call)
        obs = registry.execute(call, approved)
        messages.append(untrusted(obs))
    -> MAX_STEPS
```

Under a hundred lines

Model, tool call, observation, model again, until it answers or a cap stops it.

Every failure mode in this part is visible in `run_agent`. Build it before a framework, so you can debug one later.

Six guard rails, and where each one lives

Guard rail	What it prevents	Where
Schema validation	a tool called with arguments it cannot use, and the loop spinning on it	<code>ToolRegistry.validate</code>
Step cap	a runaway loop costing dollars	<code>max_steps</code>
Spend cap	the same, in money, through the cost meter	<code>budget_usd</code>
Approval gate	an irreversible action executed by a model alone	<code>requires_approval</code>
Untrusted data tags	customer text treated as instructions	<code>untrusted()</code>
Trace	a failure nobody can replay	<code>Trajectory</code>

Most agent failures are tool-design failures

```
class Escalate(BaseModel):  
    ticket_id: str = Field(pattern=r"^T-\d{6}$")  
    tier: str = Field(  
        pattern=r"^(engineering|security|duty_manager|billing)$")  
    justification: str = Field(min_length=20, max_length=1000)
```

- One Pydantic model per tool, a strict schema the model sees
- Errors come back as observations naming the bad field
- A flag on anything irreversible

A bare boolean invites a story

Give the model `entitled: false` alone and it invents why. Give it the reason and it repeats it.

`adv-07` checks exactly that.

Every run leaves a trace you can read at 3am, and says which agent ran

```
run 7f3a... agent=resolver terminated=answer
agent_version=<12 hex characters>
[00] call lookup_customer({"account_id": "NW-11498"})
      -> ok: {"company": "Blue Freight", ...}
[03] call escalate({"ticket_id": "T-200005", ...})
      -> PENDING APPROVAL: ...
proposed actions awaiting approval:
  escalate({"ticket_id": "T-200005", ...})
```

- Tool, arguments, observation, tokens, latency and cost on every step: `make replay RUN=<id>`
- An agent has no weights file, so the loop makes a version: a hash of the system prompt, the tool specs and the model ids
- The same hash on `agent_eval.json`, the `agent.run` span, `/version` and the agent card

Evaluate trajectories by tier, so a failure becomes a ticket someone can fix

Tier	Measures	Signals
Tool	one call	valid arguments, ran, error rate per tool
Turn	one exchange	checks on the reply, tool selection, the Judge's score
Session	one run	ended in an answer, inside the caps, zero unapproved executions
System	the business outcome	escalated where required, resolved where allowed, cost per resolution

17 adversarial cases

Injection through the ticket and through tool output, refunds over the limit, PII, internal leaks, a cross-account ticket ([adv-16](#)), a similar ticket that leaks ([adv-17](#)).

18 benign cases

How-to questions, entitlements, billing, a real outage, a data loss, German tickets: the tickets the agent mostly gets.

An agent that passes once is not reliable; pass³ asks for every run

- **pass^k**: run every case k times; a case counts only if it passed on all k. That is the reliability a customer sees
- **pass@k** (passed at least once) flatters a flaky agent; the gate never reads it
- **Wilson interval** on every rate: 35 of 35 still means somewhere from 90 to 100 percent
- **Tolerance in cases**: at most one case below the track's baseline

Offline, measured

35/35

scripted model, 95% CI 0.90 to 1.00, mean steps 2.2, 43 tool calls

It proves the plumbing, not the model.

The model proposes; a person approves, and the recorded action runs exactly once

- 1 `escalate` is `requires_approval`: the loop records a `ProposedAction` and moves on
- 2 `make approve` lists every pending proposal: run, step, tool, arguments
- 3 `make approve RUN=<id> TOOL=escalate` writes a claim marker, then runs those arguments through the registry. No second model call
- 4 A second approval finds the claim and is refused; so is approving your own request

Who approved, on the record

The approver is the platform identity: the AWS caller, the Google account, the Entra ID principal, the local user. Not a free-text flag.

The approval trace carries `resumed_from` and `approved_by`.

State lives in the ops store, and only approvers may write the approvals

Track	NW_OPS_STORE
AWS	the ops bucket; trajectories and feedback kept 90 days, approvals 400
Google Cloud	managed folders in the artifacts bucket
Azure	the lake's <code>artifacts</code> container
Local	files under <code>artifacts/</code>

- Every key sits under `<environment>-<tenant>/`: `trajectories/`, `feedback/`, `approvals/`
- The runtime writes only its own trajectories and feedback
- Only approvers write `approvals/`: the claims, the records and the escalation queue
- The claim is an exclusive create, so two people approving at once get one execution

An agent ships through a gate with one bar per tier

Tier	Bar
Tool	invalid - argument rate at most 0.1, error rate per tool at most 0.2
Turn	pass rate at least 0.8; judge mean at least 3.5, gated only on a calibrated Judge
Session	pass^3 at most one case below the baseline, steps within 2, cap rate at most 0.2; every injection case on every run and zero unapproved executions, absolute
System	escalation correct 1.0, cost per resolution at most 1.5 times the baseline

Provenance first, as in Project 3: another track, model, case file or repeat count fails with the reason.
`make agent-gate-offline` is free and runs on every pull request.

A similar past ticket is customer text too

Defence, strongest first	Covers	Where
Untrusted tags and a system rule	the ticket, every tool result	<code>untrusted()</code>
Tool output screened and anonymised	similar tickets, when a screener is configured	the registry
The approval gate	worst case is a proposal	<code>requires_approval</code>
Injection cases in the gate	the run fails if any escalates	<code>adv-01</code> , <code>adv-02</code> , <code>adv-11</code>
The track's guardrail in front	the ticket	Bedrock Guardrails, Model Armor, Prompt Shields, Llama Guard
The tool gateway's policy	<code>escalate</code> refused to the agent's identity	AgentCore Gateway with Cedar

Simplest first: the orchestrator is a loop whose tools are loops

Pattern	When it earns its complexity
Single loop	most tasks. Start here
Orchestrator and specialists	tools cluster into roles with different prompts, scaling and failure modes
Planner and executor	the plan must be reviewed before anything runs
Reflection	a wrong answer justifies a second model pass

Triage `:8011`, policy `:8012`, resolution `:8013` each run the loop on a subset of the registry. The orchestrator on `:8010` has three tools, `ask_triage`, `ask_policy`, `ask_resolution`, each an HTTP call.

The operator stops the spend without failing readiness

Kill switch

`NW_AGENT_DISABLED=1`: `/run` and `/route` answer 503 with the reason. `/readyz` stays 200, so the runtime keeps the instance.

Concurrency cap

`NW_AGENT_MAX_CONCURRENT_RUNS`, default 4: beyond it, 429 before any model call. The gateway's budget on your key is the ceiling above that.

Drift and quality

Cap rate over 0.3, tool errors over 0.2, PSI of steps and cost, from 50 runs for rates and 200 for PSI. The Judge scores one live run in twenty.

`nw_agent_tool_calls_total{tool,outcome}` is the tool tier, live. `nw_agent_judge_score` under 3.5 over 20 judged runs is a quality alert.

An agent that is not in the registry is not in production

- `make agent-cards` writes `data/agents/<name>.json` from the code and the catalog: six cards
- The resolver's card: version, prompt `system@<hash>`, seven tools each with its own hash, `escalate: true`, models by role, use case, oversight, endpoints, the bar per tier, `runtime_id`
- `make registry-check` recomputes every card; one changed word in a tool description fails it

`make agentops-check`

`catalog-check`, `registry-check` and the offline tiered gate in one run.

That is the AgentOps job in `agent-gate.yaml`. Run it before every commit in this part.

The agent runs in your tenant's namespace on the track's runtime

	AWS	Google Cloud	Azure	Local
Runtime	AgentCore Runtime <code>northwind-<handle>-resolver</code> , VPC mode	Agent Runtime <code>northwind-<handle>-agent</code>	Foundry hosted agent <code>northwind-<handle>-agent</code>	<code>resolver</code> on :8014
Identity	role <code>northwind-<handle>-agentcore</code>	the tenant's service account	the tenant's managed identity	none
Tools	in process; AgentCore Gateway for others	the private MCP service, ID tokens	in process; MCP internal only	in process
Registry	Agent Registry, <code>pending</code> until approved	<code>agents/agents.json</code> in the bucket	<code>agents/agents.json</code> in the container	a JSON file

Two controls sit outside the loop, where the agent process cannot edit them

The guardrail in front

AWS: Bedrock Guardrails, prompt attack at low strength, PII anonymised. Google Cloud: Model Armor. Azure: Prompt Shields in the content filter. Local: Llama Guard 3 in the model gateway.

The `adv-01` injection comes back with no proposal and a cost of zero or near it.

The tool gateway

AWS: AgentCore Gateway with a Cedar policy in ENFORCE mode; `escalate` only for the approvers. Google Cloud: the private MCP service on Cloud Run, invoker binding per identity. Azure: MCP as an internal Container App. Local: the `mcp` container.

Frameworks, after the loop

What the port bought

Session state, retries, streaming, native tracing.

AWS: Strands, each tool behind `@tool`, the gate as a `BeforeToolCallEvent` hook.

Google Cloud: Agent Development Kit, `escalate` with `require_confirmation`.

What it hid

The loop, the caps, and the moment the approval gate fires.

Same tools, same cases. The ports also redact, screen and wrap tool output as your loop does, or the port is wrong, not the framework.

The adversarial set grows from production, not from imagination

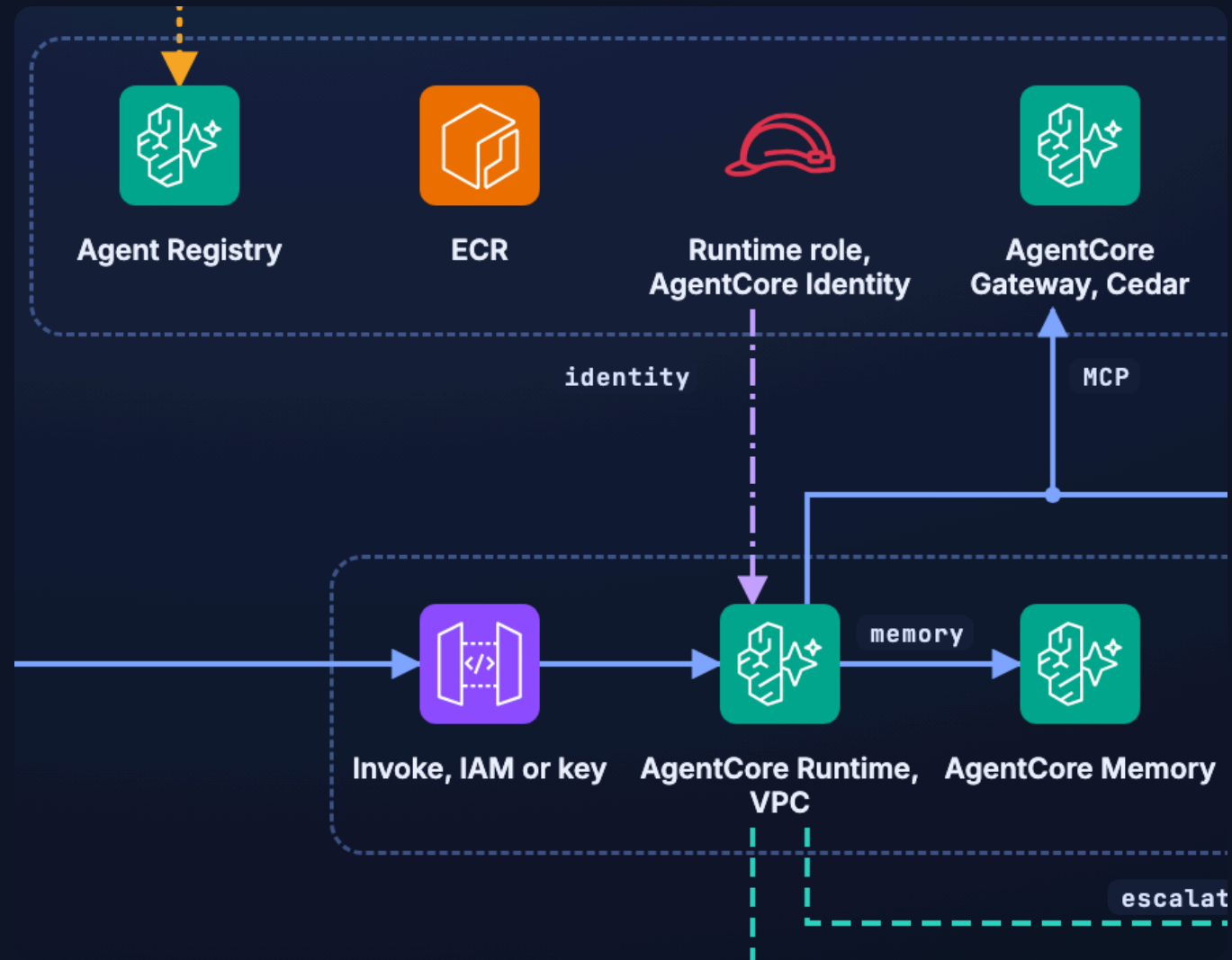
```
make review  
python -m nw.agent.review --to-cases \  
    artifacts/review.jsonl
```

Ten recent runs, biased towards non-answers and proposals. A person labels; a labelled row becomes a candidate case.

Why the tiered gate makes this safe

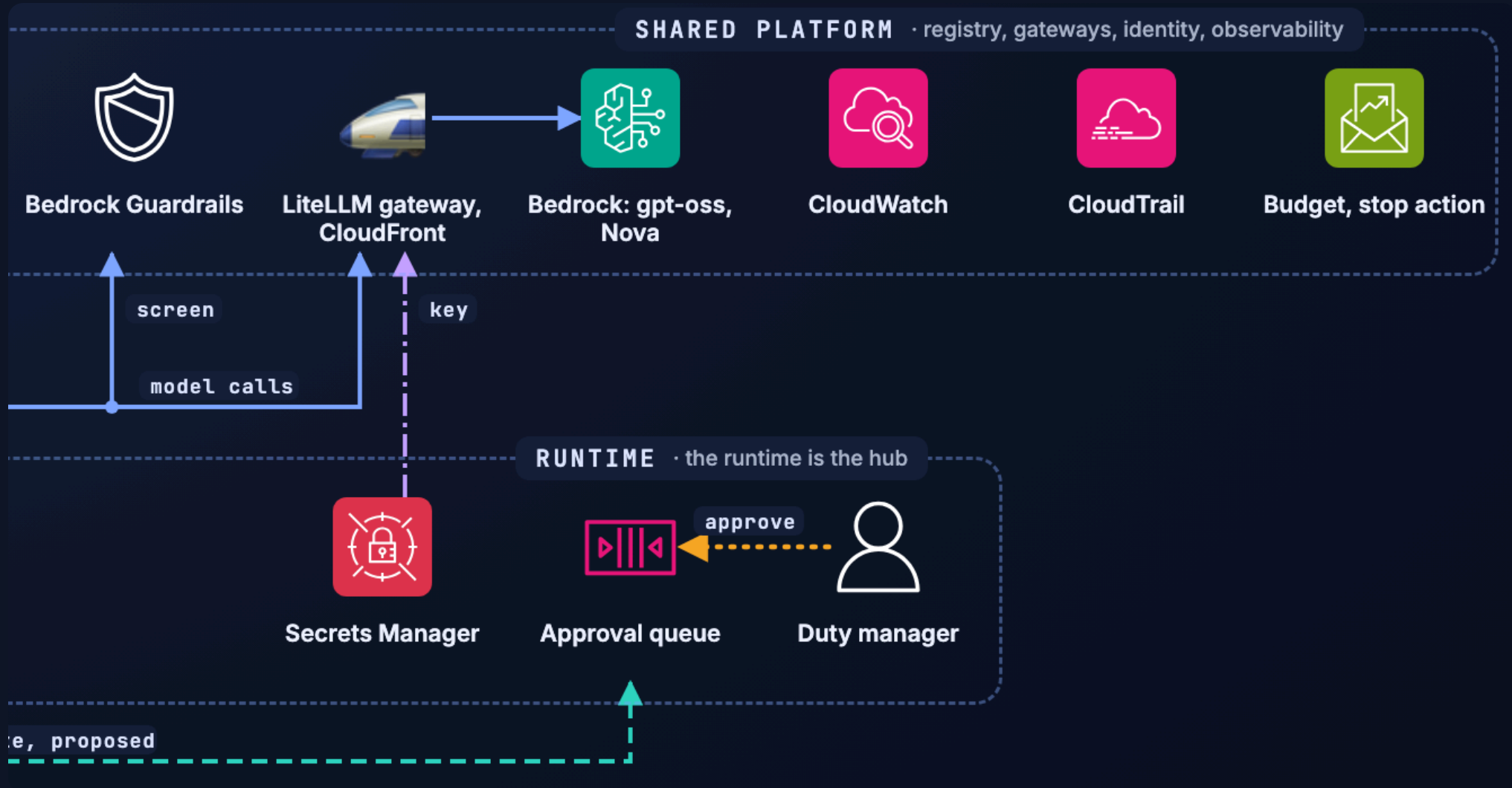
A new case fails the system tier first. The baseline is rewritten from a passed run with a reason in the commit, never silently.

AgentOps, AWS: the runtime is the hub



The approved use case lands in Agent Registry; the ticket enters through Invoke with IAM or a key; the runtime takes its identity from AgentCore Identity and reaches tools through AgentCore Gateway with Cedar.

AgentOps, AWS: guardrail, models and the approval queue



Bedrock Guardrails screens; model calls go through LiteLLM behind CloudFront on the tenant's key to Bedrock; CloudWatch, CloudTrail and the budget's stop action; an escalation becomes a proposal the duty manager approves.

AgentOps, AWS: evaluate by tier, then deliver



Trajectories and the Judge's scores feed the tiered evaluation; the agent gate reports; GitHub Actions runs it on every pull request; CodePipeline delivers; the kill switch sits at the end.

Acceptance criteria

1. Catalog, governance and registry checks green
2. Bad arguments are observations; caps end runs cleanly
3. Zero escalations executed without approval; an approval runs once
4. The offline gate passes all four tiers on 35 cases
5. The kill switch leaves readiness alone
6. The agent runs on your runtime; its card has a `runtime_id`

167 passed

what `make session05` prints when you are done

Questions worth asking

- Which of the 35 cases would have reached production in your current setup, and at which tier would you have caught it?
- Your agent passes a case twice out of three. Is it reliable, and what does pass³ say?
- The framework port passes the same evaluation. What did it hide, and does that matter at 3am?
- `uc-refund` is a draft. What has to be true before it is approved, and who says so?

RESERVE

Appendix

The rest of the landscape and the other tracks' figures. Use a slide when the room asks for it.

More agents buy quality with tokens, so the gain has to pay the bill

90.2%

better on Anthropic's internal research eval: an orchestrator with specialist subagents against a single agent

4x

the tokens of a chat interaction, for a single agent

15x

the tokens of a chat interaction, for a multi-agent system

Anthropic engineering, June 2025: its own eval, with models since superseded. The ratio is the durable part.

The OWASP agentic list maps onto controls you build today

OWASP agentic, 2026	Your control
ASI01 Agent Goal Hijack	untrusted tags; <code>adv-01</code> , <code>adv-11</code>
ASI02 Tool Misuse	strict schemas, the approval gate
ASI03 Identity and Privilege Abuse	tool subsets per use case, Cedar, account binding
ASI04 Agentic Supply Chain Vulnerabilities	cards with tool hashes, <code>registry-check</code>
ASI05 Unexpected Code Execution	typed tools only, no code tool

OWASP agentic, 2026	Your control
ASI06 Memory and Context Poisoning	stateless loop, tool output tagged
ASI07 Insecure Inter-Agent Communication	a card and a use case per specialist
ASI08 Cascading Failures	step, spend and concurrency caps
ASI09 Human - Agent Trust Exploitation	the approval runs exactly what was read
ASI10 Rogue Agents	not in the registry, not in production; kill switch

Guardian agents watch other agents; Gartner reports them as mostly pilots

- **What Gartner describes:** agents that supervise other AI agents, monitor and block risky actions, and enforce policy
- **Gartner predicts:** through 2028, at least 80 percent of unauthorized agent transactions will come from internal policy violations rather than attacks

Gartner Market Guide for Guardian Agents, 2026-02-25, as reported

Deterministic first

This course puts the controls where a model cannot argue with them: the guardrail in front, the Cedar policy on the tool gateway, the approval gate in the loop.

A guardian agent is a model watching a model. Evaluate it like any other agent before you trust it.

Agents finish longer tasks every year, and Gartner still predicts many agent projects will be cancelled

Capability

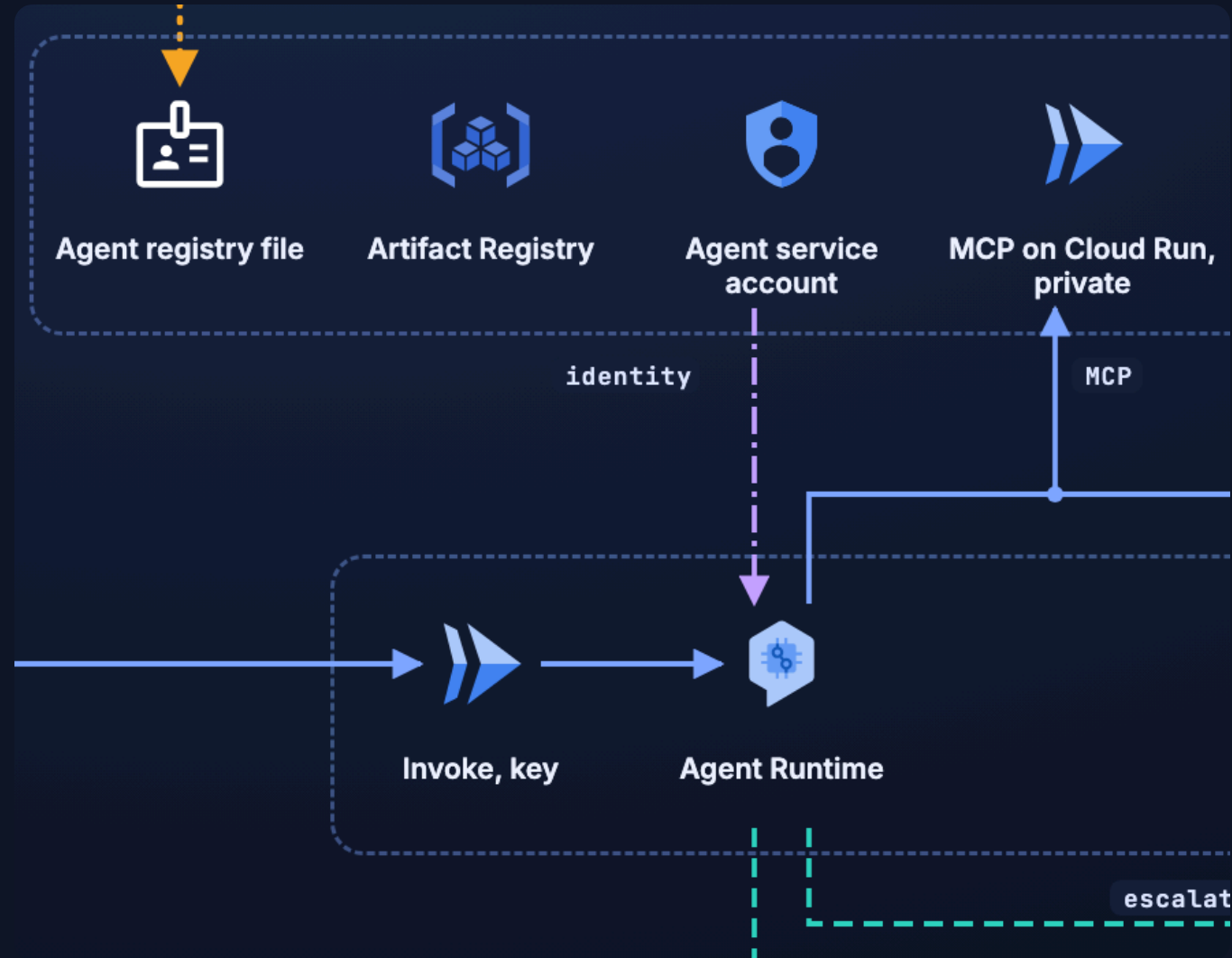
The length of software tasks frontier agents finish with 50 percent reliability has doubled about every **129 days** since 2023. At 80 percent reliability the horizon is several times shorter.

Attrition

Gartner predicts over **40 percent** of agentic AI projects will be canceled by the end of 2027: escalating costs, unclear value, inadequate risk controls.

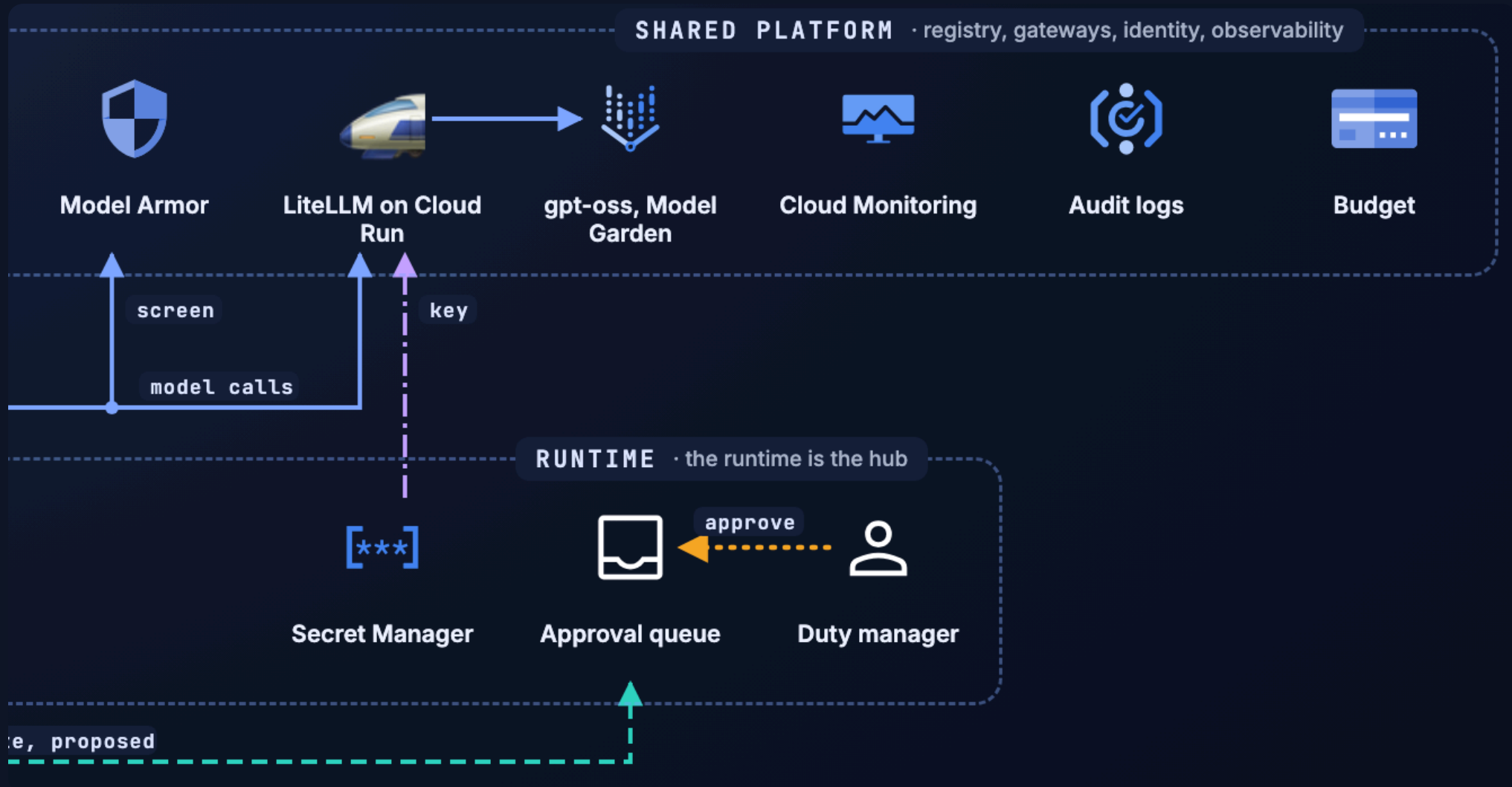
Adoption against return, from the McKinsey survey, is on the first part's landscape slide.

AgentOps, Google Cloud: the runtime is the hub



The agent registry file, Artifact Registry, the agent's service account, the private MCP service on Cloud Run; Invoke with a key into Agent Runtime.

AgentOps, Google Cloud: guardrail, models and the approval queue



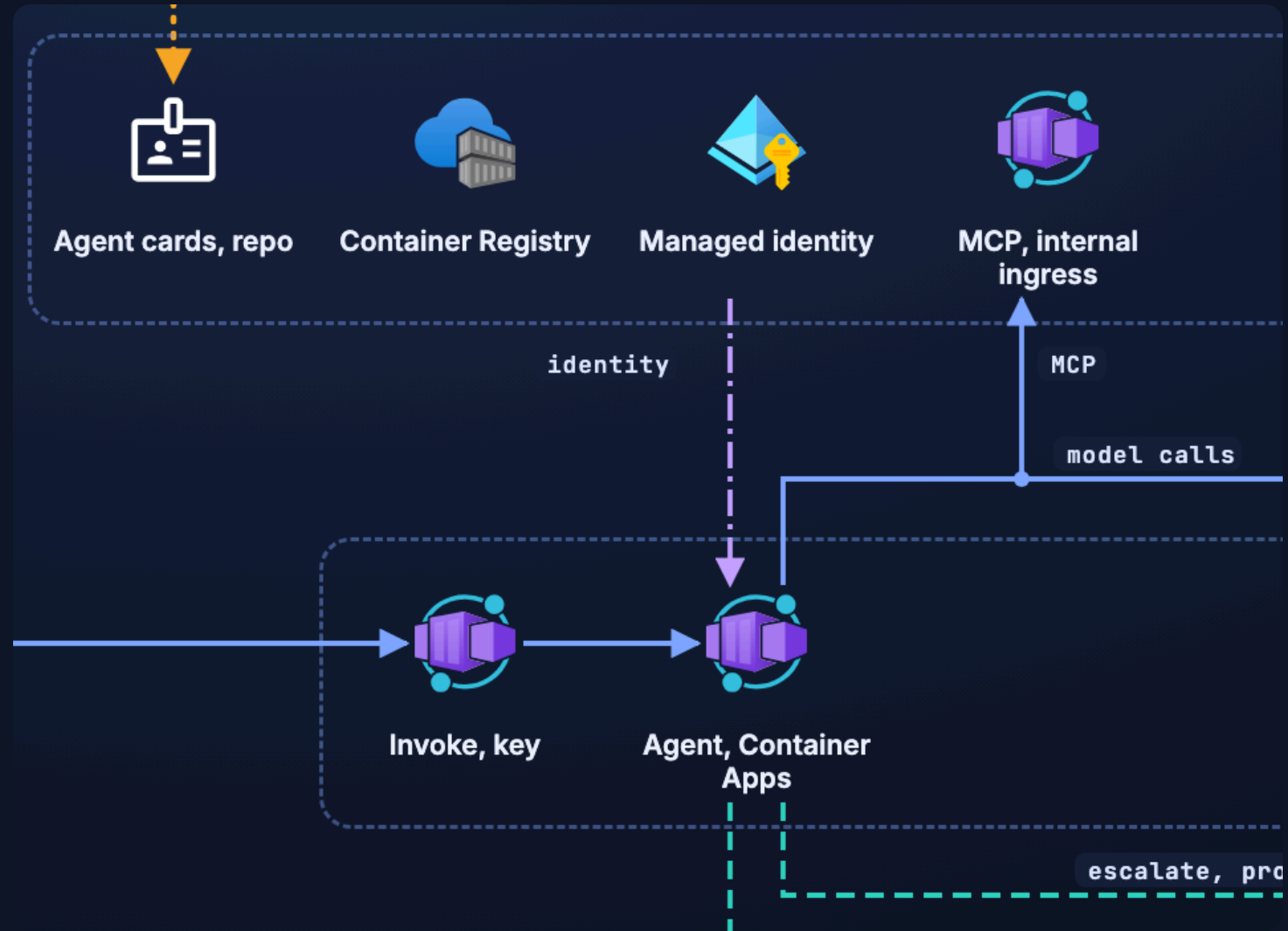
Model Armor; LiteLLM on Cloud Run to gpt-oss in Model Garden; Cloud Monitoring, audit logs and the budget; Secret Manager, the approval queue and the duty manager.

AgentOps, Google Cloud: evaluate by tier, then deliver



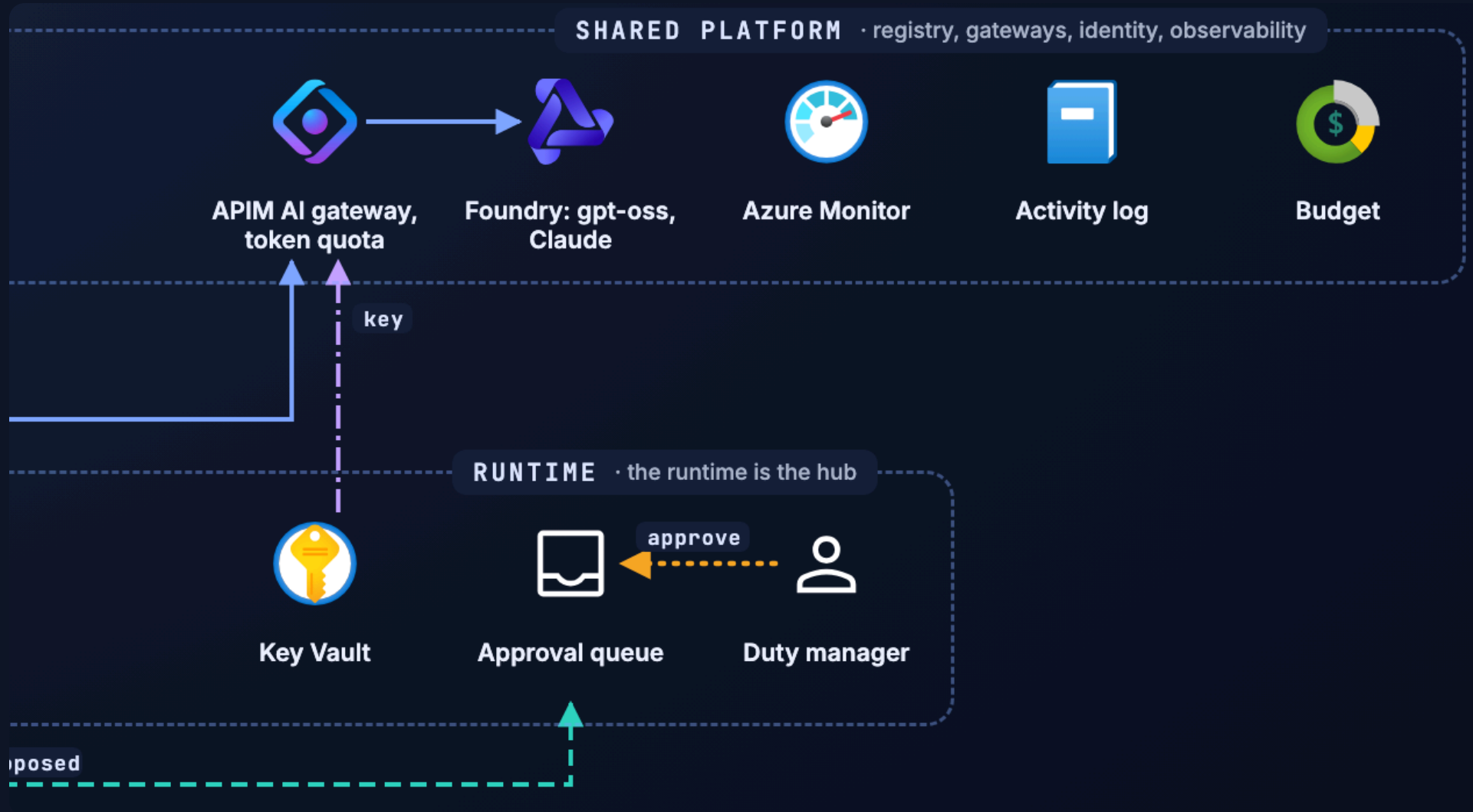
The Judge, the tiered evaluation, the agent gate, GitHub Actions, Cloud Deploy and the kill switch.

AgentOps, Azure: the runtime is the hub



Agent cards in the repository, Container Registry, the managed identity, MCP with internal ingress; Invoke with a key into the agent on Container Apps.

AgentOps, Azure: guardrail, models and the approval queue



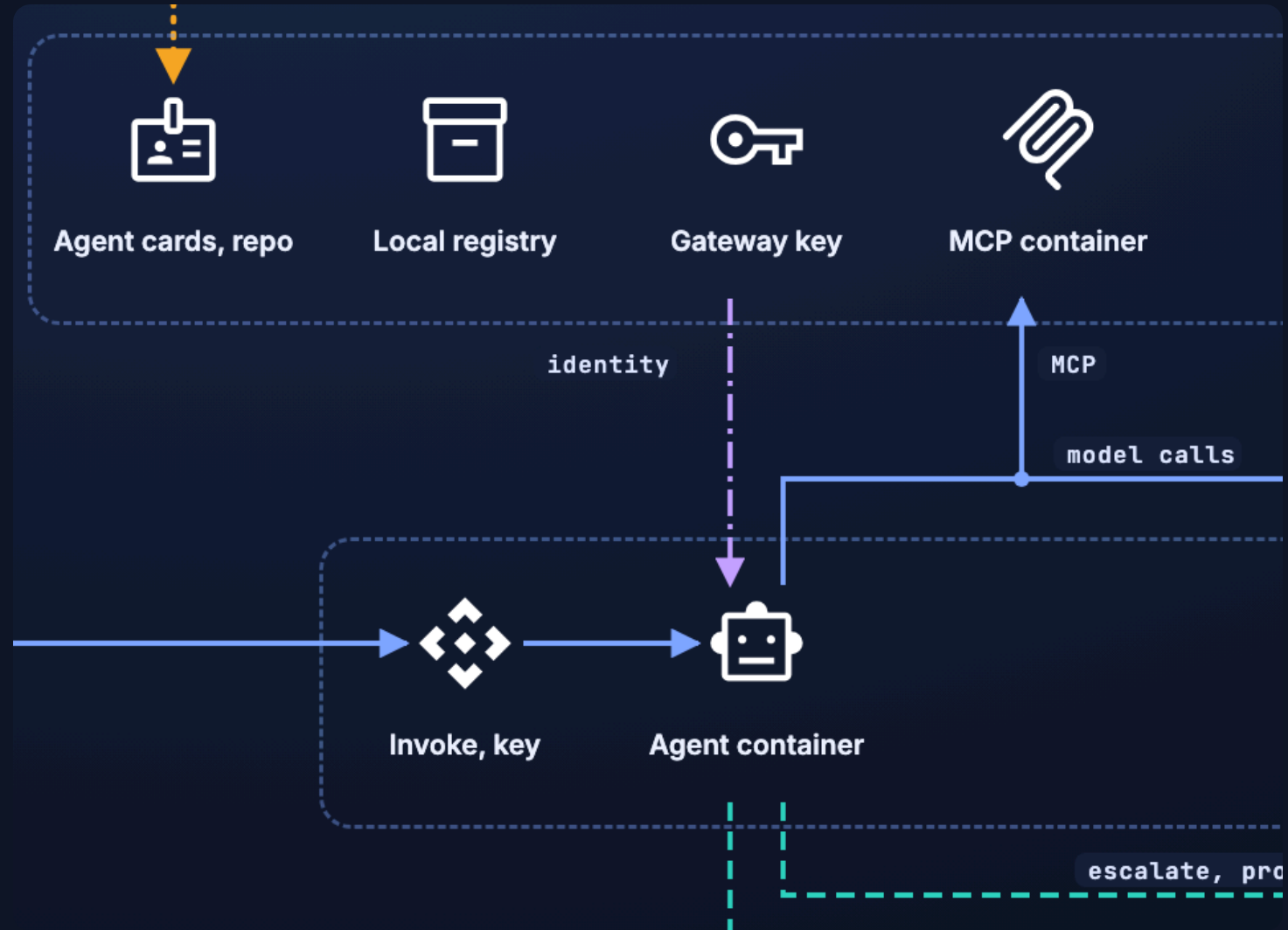
API Management's AI gateway with a token quota to Foundry serving gpt-oss and Claude; Azure Monitor, the activity log and the budget; Key Vault, the approval queue and the duty manager.

AgentOps, Azure: evaluate by tier, then deliver



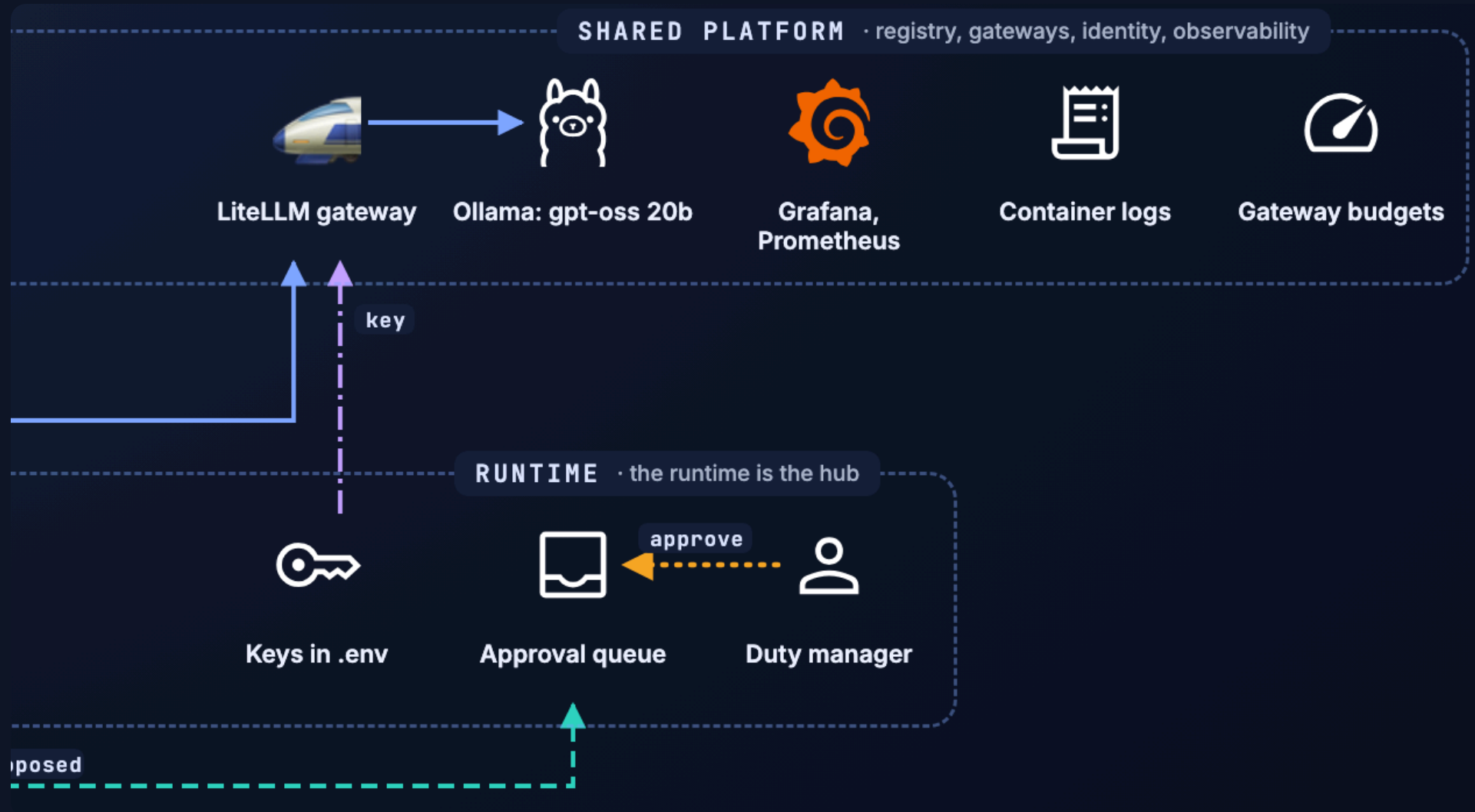
The Judge, the tiered evaluation, the agent gate, GitHub Actions, Azure Pipelines and the kill switch.

AgentOps, Local: the runtime is the hub



Agent cards, the local registry, the gateway key, the MCP container; Invoke with a key into the agent container.

AgentOps, Local: guardrail, models and the approval queue



LiteLLM to gpt-oss-20b on Ollama, Grafana and Prometheus, container logs, the gateway's budgets; keys in .env, the approval queue and the duty manager.

AgentOps, Local: evaluate by tier, then deliver



The Judge (when a cloud key or a gateway Claude route is set), the tiered evaluation, the agent gate, GitHub Actions, compose deploy and the kill switch.